

Développement Web PHP

Architecture MVC

Moteurs de **Template**

Gestionnaire de **Dépendances**

Composer · Twig · PHP



Table des matières

1	Architecture MVC	3
1.1	Qu'est-ce que le patron MVC ?	3
1.1.1	Historique et motivation	3
1.2	Les apports du MVC	4
1.3	Les trois composants et leurs rôles	4
1.3.1	Le Modèle (Model)	4
1.3.2	La Vue (View)	5
1.3.3	Le Contrôleur (Controller)	5
1.4	Le flux de traitement d'une requête	6
1.5	Mise en œuvre complète en PHP	7
1.5.1	Structure de fichiers recommandée	7
1.5.2	Le Front Controller et le routeur	7
1.5.3	Le Repository (couche d'accès aux données)	9
2	Moteur de template	11
2.1	Définition et motivation	11
2.2	Fonctionnalités d'un moteur de template	11
2.2.1	Affichage de variables	11
2.2.2	Structures de contrôle	12
2.2.3	Héritage de templates (Layout)	12
2.2.4	Inclusion de sous-templates	13
2.2.5	Variables globales et fonctions Twig	14
2.3	Intégrer Twig dans une architecture MVC	14
2.3.1	Installation via Composer	14
2.3.2	Configuration de Twig	14
2.3.3	Contrôleur avec Twig	15
3	Gestionnaire de dépendances — Composer	17
3.1	Rôle d'un gestionnaire de dépendances	17
3.2	Structure d'un projet Composer	17
3.2.1	Le fichier <code>composer.json</code>	17
3.2.2	Le fichier <code>composer.lock</code>	18
3.2.3	Le répertoire <code>vendor/</code>	18
3.3	Commandes essentielles de Composer	19
3.3.1	Initialiser un projet	19
3.3.2	Installer les dépendances	19
3.3.3	Mettre à jour les dépendances	19
3.3.4	Supprimer une dépendance	20
3.3.5	Autoloading et informations	20
3.3.6	Syntaxe des contraintes de version	20
3.4	L'autoloading PSR-4	21

4	Synthèse : Projet PHP complet MVC + Twig + Composer	22
4.1	Architecture globale	22
4.2	Récapitulatif des commandes Composer	22
4.3	Récapitulatif des fonctionnalités Twig	23

1 Architecture MVC

🎯 Objectifs

- [1] Rappeler les apports de l'architecture MVC dans le développement d'une application
- [2] Expliquer le rôle des composants de l'architecture MVC et leurs interactions
- [4] Mettre en œuvre l'architecture MVC pour développer une application PHP complexe

1.1 Qu'est-ce que le patron MVC ?

📖 Définition

MVC (Model-View-Controller) est un **patron d'architecture logicielle** qui sépare une application en trois couches distinctes ayant chacune une responsabilité unique. Introduit par Trygve Reenskaug en 1979, il est aujourd'hui l'un des patrons les plus utilisés dans le développement web.

L'idée centrale du MVC repose sur le **principe de séparation des préoccupations** (*Separation of Concerns*) : chaque partie du code n'a qu'une seule raison de changer. Un bug visuel ne touche que la Vue, un bug métier ne touche que le Modèle.

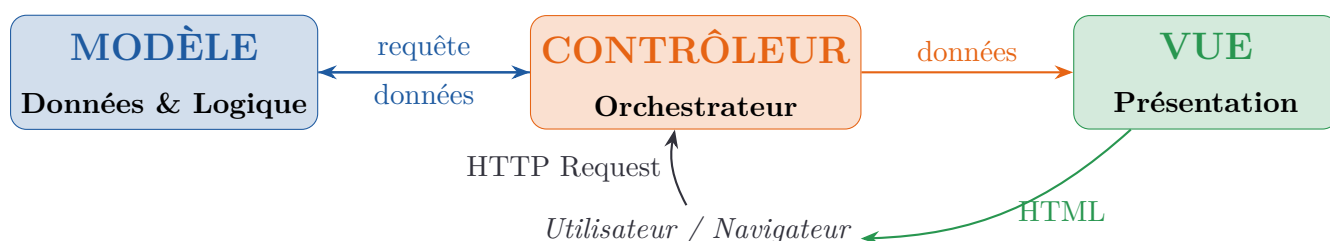
1.1.1. Historique et motivation — Avant le MVC, les applications PHP étaient souvent écrites de façon *procédurale* : HTML, SQL et PHP mélangés dans un seul fichier. Ce style dit **spaghetti code** posait de nombreux problèmes :

- Maintenance difficile : toute modification pouvait casser plusieurs fonctionnalités
- Test impossible : on ne pouvait pas tester la logique métier sans rendu HTML
- Travail en équipe délicat : les intégrateurs HTML modifiaient le même fichier que les développeurs PHP
- Réutilisation nulle : le code d'accès aux données était noyé dans la présentation

1.2 Les apports du MVC

Apport	Description
Séparation des responsabilités	Chaque couche a un rôle précis. On peut modifier l’affichage sans toucher à la logique.
Testabilité	Le Modèle (logique métier) peut être testé unitairement, sans navigateur ni HTML.
Maintenabilité	Le code est organisé et prévisible ; un nouveau développeur sait où chercher.
Réutilisabilité	Un même Modèle peut servir une Vue HTML, une API JSON, ou une sortie PDF.
Travail en équipe	Les intégrateurs travaillent sur les Vues, les développeurs sur les Modèles.
Évolutivité	Ajouter une fonctionnalité ne nécessite pas de ré-écrire l’application entière.

1.3 Les trois composants et leurs rôles



1.3.1. Le Modèle (Model) — Le Modèle est le **cœur métier** de l’application. Il est totalement indépendant de la couche de présentation.

- Représente les **entités** du domaine (User, Article, Produit...)
- Contient les **règles métier** (calcul du prix TTC, validation d’un email...)
- Gère l’accès aux **données** (base de données, API, fichiers...)
- **Ne connaît pas** la Vue ni le Contrôleur

Exemple — Modèle User

```

1 <?php
2 namespace App\Model;
3
4 class User {
5     private int $id;
6     private string $name;
7     private string $email;
8
9     public function __construct(int $id, string $name, string
10         $email) {
11         $this->id = $id;
12         $this->name = $name;
13     }
14 }
  
```

```

12     $this->email = $email;
13 }
14
15 // Règle métier : validation du format email
16 public function isValidEmail(): bool {
17     return filter_var($this->email, FILTER_VALIDATE_EMAIL)
18         !== false;
19 }
20
21 public function getName(): string { return $this->name; }
22 public function getEmail(): string { return $this->email; }
23 public function getId(): int      { return $this->id; }

```

1.3.2. La Vue (View) — La Vue est responsable uniquement de **l’affichage**. Elle reçoit des données du Contrôleur et les présente à l’utilisateur.

- Ne contient **aucune logique métier**
- Ne fait **aucun accès base de données**
- Peut contenir une logique d’affichage minimale (boucles, conditions d’affichage)
- Souvent un fichier HTML avec quelques instructions PHP ou de template

Exemple — Vue users/list.php

```

1 <!DOCTYPE html>
2 <html lang="fr">
3 <head><title>Liste des utilisateurs</title></head>
4 <body>
5     <h1>Utilisateurs</h1>
6     <ul>
7         <?php foreach ($users as $user): ?>
8             <li>
9                 <?= htmlspecialchars($user->getName()) ?>
10                 <?= htmlspecialchars($user->getEmail()) ?>
11             </li>
12         <?php endforeach; ?>
13     </ul>
14 </body>
15 </html>

```

1.3.3. Le Contrôleur (Controller) — Le Contrôleur est le **chef d’orchestre**. Il reçoit la requête HTTP, coordonne le Modèle et la Vue, puis renvoie une réponse.

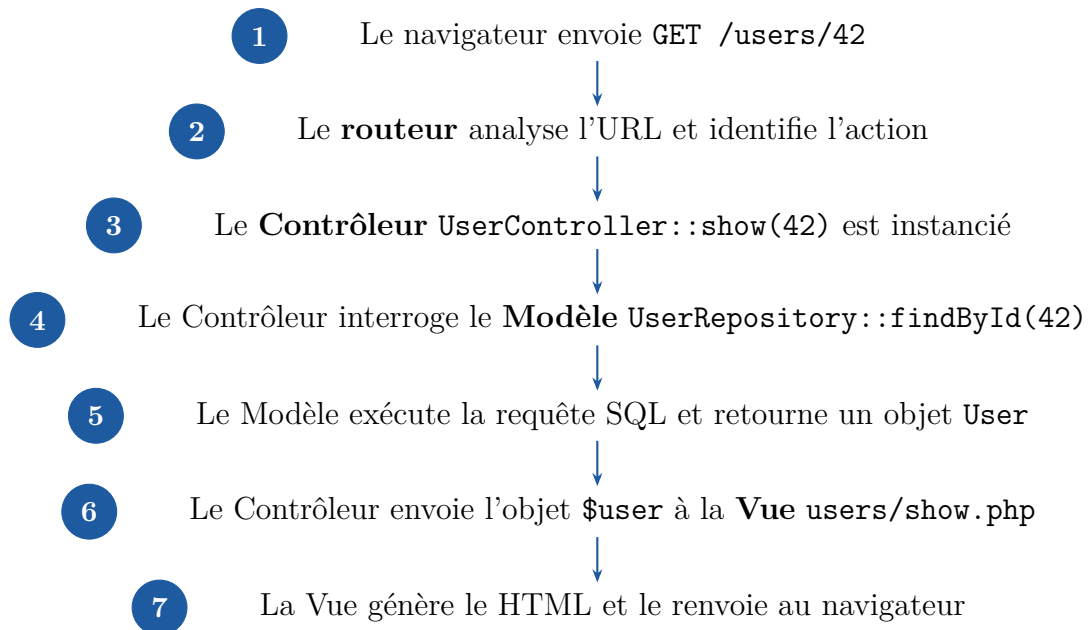
- Reçoit et analyse la **requête HTTP**
- Appelle les Modèles pour récupérer/modifier des données
- Sélectionne et envoie les données à la **Vue appropriée**
- Ne contient pas de logique métier complexe (délègue au Modèle)
- Ne contient pas de code SQL (délègue au Modèle)

</> Exemple — Contrôleur UserController

```
1 <?php
2 namespace App\Controller;
3
4 use App\Model\UserRepository;
5
6 class UserController {
7     private UserRepository $repository;
8
9     public function __construct(UserRepository $repository) {
10         $this->repository = $repository;
11     }
12
13     // Action : afficher la liste des utilisateurs
14     public function list(): void {
15         // 1. Récupérer les données via le Modèle
16         $users = $this->repository->findAll();
17
18         // 2. Passer les données à la Vue
19         require __DIR__ . '/../views/users/list.php';
20     }
21
22     // Action : afficher un utilisateur par son ID
23     public function show(int $id): void {
24         $user = $this->repository->findById($id);
25
26         if ($user === null) {
27             http_response_code(404);
28             require __DIR__ . '/../views/errors/404.php';
29             return;
30         }
31
32         require __DIR__ . '/../views/users/show.php';
33     }
34 }
```

1.4 Le flux de traitement d'une requête

Voici le déroulement type d'une requête GET `/users/42` dans une application MVC :



1.5 Mise en œuvre complète en PHP

1.5.1. Structure de fichiers recommandée —

```
mon-projet/  
  public/                                # Seul dossier exposé sur le web  
    index.php                            # Point d'entrée unique (Front  
Controller)  
  src/  
    Controller/                          # Contrôleurs  
      UserController.php  
    Model/                               # Modèles et entités  
      User.php  
      UserRepository.php  
    Router.php                           # Routeur  
  views/                                 # Vues (templates PHP)  
    users/  
      list.php  
      show.php  
    layout/  
      base.php  
  composer.json
```

1.5.2. Le Front Controller et le routeur —

⚠ Point important

Le **Front Controller** (souvent public/index.php) est le **point d'entrée unique** de l'application. Toutes les requêtes HTTP passent par lui, qui délègue au routeur.

</> public/index.php — Front Controller

```
1 <?php
2 // Chargement de l'autoloader Composer
3 require_once __DIR__ . '/../vendor/autoload.php';
4
5 use App\Router;
6
7 // Instanciation du routeur et dispatch
8 $router = new Router();
9 $router->get('/users', ['App\Controller\UserController', '
    list']);
10 $router->get('/users/{id}', ['App\Controller\UserController', '
    show']);
11
12 $router->dispatch($_SERVER['REQUEST_URI'], $_SERVER['
    REQUEST_METHOD']);
```

</> src/Router.php — Routeur simplifié

```
1 <?php
2 namespace App;
3
4 class Router {
5     private array $routes = [];
6
7     public function get(string $path, array $handler): void {
8         $this->routes['GET'][$path] = $handler;
9     }
10
11     public function dispatch(string $uri, string $method): void {
12         $uri = strtok($uri, '?'); // Ignorer les query strings
13
14         foreach ($this->routes[$method] ?? [] as $pattern =>
            $handler) {
15             // Convertir {id} en groupe de capture regex
16             $regex = '#^' . preg_replace('/\{(\w+)\}/', '(\d+)',
                $pattern) . '$#';
17
18             if (preg_match($regex, $uri, $matches)) {
19                 array_shift($matches); // Enlever le match
                    complet
20                 [$class, $action] = $handler;
21                 $controller = new $class(/* injection des
                    dépendances */);
22                 $controller->$action(...$matches);
23                 return;
24             }
25         }
26
27         http_response_code(404);
28         echo "404      Page non trouv e";
29     }
30 }
```

1.5.3. Le Repository (couche d'accès aux données) —



Définition

Le **Repository** est un patron complémentaire au MVC. Il isole le code d'accès à la base de données dans une classe dédiée, gardant ainsi le Modèle (entité) pur de toute logique SQL.

src/Model/UserRepository.php

```
1 <?php
2 namespace App\Model;
3
4 use PDO;
5
6 class UserRepository {
7     private PDO $pdo;
8
9     public function __construct(PDO $pdo) {
10         $this->pdo = $pdo;
11     }
12
13     public function findAll(): array {
14         $stmt = $this->pdo->query('SELECT * FROM users ORDER BY
15             name');
16         $rows = $stmt->fetchAll(PDO::FETCH_ASSOC);
17         return array_map(fn($row) => new User(
18             $row['id'], $row['name'], $row['email']
19         ), $rows);
20     }
21
22     public function findById(int $id): ?User {
23         $stmt = $this->pdo->prepare(
24             'SELECT * FROM users WHERE id = :id'
25         );
26         $stmt->execute([':id' => $id]);
27         $row = $stmt->fetch(PDO::FETCH_ASSOC);
28
29         if (!$row) return null;
30
31         return new User($row['id'], $row['name'], $row['email']);
32     }
33
34     public function save(User $user): void {
35         $stmt = $this->pdo->prepare(
36             'INSERT INTO users (name, email) VALUES (:name, :
37                 email)'
38         );
39         $stmt->execute([
40             ':name' => $user->getName(),
41             ':email' => $user->getEmail(),
42         ]);
43     }
44 }
```

**Bonne pratique**

Injection de dépendances : Passez le PDO en paramètre du constructeur plutôt que de le créer à l'intérieur du Repository. Cela facilite les tests unitaires (on peut injecter un mock) et respecte le principe d'inversion de dépendances.

2 Moteur de template

🎯 Objectifs

- [1] Définir ce qu'est un moteur de template
- [2] Citer les fonctionnalités proposées par un moteur de template
- [3] Utiliser un moteur de template pour développer une application PHP
- [4] Intégrer un moteur de template dans une architecture MVC

2.1 Définition et motivation

📖 Définition

Un **moteur de template** (ou moteur de rendu) est une bibliothèque qui permet de séparer le code HTML du code PHP en utilisant une **syntaxe dédiée** plus lisible. Il compile les templates en PHP natif optimisé et ajoute des fonctionnalités absentes du PHP brut.

```
1 <?php if (!empty($users)): ?>
2   <?php foreach ($users as $user): ?>
3     <li><?= htmlspecialchars($user->getName()) ?></li>
4   <?php endforeach; ?>
5 <?php else: ?>
6   <li>Aucun utilisateur</li>
7 <?php endif; ?>
```

Listing 1 – Vue PHP natif — peu lisible

Listing 2 – Vue Twig — syntaxe claire

```
1 {% if users %}
2   {% for user in users %}
3     <li>{{ user.name }}</li>
4   {% endfor %}
5 {% else %}
6   <li>Aucun utilisateur</li>
7 {% endif %}
```

2.2 Fonctionnalités d'un moteur de template

2.2.1. Affichage de variables —

Listing 3 – Affichage et filtres Twig

```

1  {# Afficher une variable (chappement automatique) #}
2  <p>Bonjour {{ user.name }}</p>
3
4  {# Filtres : modifier l'affichage sans changer les données #}
5  <p>{{ user.name | upper }}</p>           {# JEAN DUPONT #}
6  <p>{{ price | number_format(2, ',', ' ') }}</p>
7  <p>{{ article.date | date('d/m/Y') }}</p>
8  <p>{{ description | truncate(100) }}</p>
9
10 {# Désactiver l'escaping (HTML de confiance uniquement) #}
11 {{ article.content | raw }}

```

Listing 4 – Conditions et boucles

2.2.2. Structures de contrôle —

```

1  {# Condition #}
2  {% if user.isAdmin %}
3      <span class="badge">Admin</span>
4  {% elseif user.isModerator %}
5      <span class="badge">Modérateur</span>
6  {% else %}
7      <span class="badge">Membre</span>
8  {% endif %}
9
10 {# Boucle avec accès à l'index et vérification du dernier élément #}
11 {% for product in products %}
12     <tr class="{{ loop.index is odd ? 'odd' : 'even' }}">
13         <td>{{ loop.index }}. {{ product.name }}</td>
14         <td>{{ product.price }}</td>
15     </tr>
16 {% else %}
17     <tr><td colspan="2">Aucun produit disponible</td></tr>
18 {% endfor %}

```

2.2.3. Héritage de templates (Layout) —

⚠ Point important

L'**héritage de templates** est la fonctionnalité la plus puissante de Twig. Elle permet de définir un gabarit (*layout*) commun à toutes les pages, évitant la duplication du HTML de base (header, footer, navbar...).

Listing 5 – templates/layout/base.html.twig — gabarit de base

```

1  <!DOCTYPE html>
2  <html lang="fr">
3  <head>
4      <meta charset="UTF-8">
5      <title>{% block title %}Mon Application{% endblock %}</title>
6      <link rel="stylesheet" href="/css/app.css">
7      {% block stylesheets %}{% endblock %}
8  </head>
9  <body>

```

```

10 <nav>
11   <a href="/">Accueil</a>
12   <a href="/users">Utilisateurs</a>
13 </nav>
14
15 <main class="container">
16   {% block content %}
17   {# Les pages enfants remplissent ce bloc #}
18   {% endblock %}
19 </main>
20
21 <footer>
22   <p>&copy; {{ "now" | date("Y") }} Mon Application</p>
23 </footer>
24
25 {% block javascripts %}{% endblock %}
26 </body>
27 </html>

```

Listing 6 – templates/users/list.html.twig — page héritant du layout

```

1 {% extends 'layout/base.html.twig' %}
2
3 {% block title %}Liste des utilisateurs{% endblock %}
4
5 {% block content %}
6   <h1>Utilisateurs ({{ users | length }})</h1>
7
8   <table class="table">
9     <thead>
10      <tr><th>Nom</th><th>Email</th><th>Actions</th></tr>
11    </thead>
12    <tbody>
13      {% for user in users %}
14        <tr>
15          <td>{{ user.name }}</td>
16          <td>{{ user.email }}</td>
17          <td>
18            <a href="/users/{{ user.id }}">Voir</a>
19          </td>
20        </tr>
21      {% endfor %}
22    </tbody>
23  </table>
24 {% endblock %}

```

Listing 7 – Inclusion et macros

2.2.4. Inclusion de sous-templates —

```

1 {# Inclure un template partiel (carte produit) #}
2 {% include 'components/product-card.html.twig' with { product: product
3   } %}
4
5 {# Macro : composant r utilisable (comme une fonction) #}
6 {% macro input(name, value, type = 'text') %}
7   <input type="{{ type }}" name="{{ name }}" value="{{ value | e }}">
8 {% endmacro %}

```

```

8
9 {# Utilisation de la macro #}
10 {{ _self.input('email', user.email, 'email') }}

```

Listing 8 – Fonctions et tests Twig

2.2.5. Variables globales et fonctions Twig —

```

1 {# Tests #}
2 {% if user.email is not empty %}...{% endif %}
3 {% if users is iterable %}...{% endif %}
4 {% if id is divisible by(2) %}...{% endif %}
5
6 {# Fonctions built-in #}
7 {{ range(1, 5) | join(', ') }}      {# 1, 2, 3, 4, 5 #}
8 {{ max(3, 7, 2) }}                 {# 7 #}
9 {{ dump(user) }}                   {# Debug : affiche la variable #}

```

2.3 Intégrer Twig dans une architecture MVC

2.3.1. Installation via Composer —

```
composer require twig/twig
```

2.3.2. Configuration de Twig —

</> src/TwigFactory.php — Configuration de Twig

```

1 <?php
2 namespace App;
3
4 use Twig\Environment;
5 use Twig\Loader\FilesystemLoader;
6
7 class TwigFactory {
8     public static function create(): Environment {
9         $loader = new FilesystemLoader(__DIR__ . '/../templates')
10             ;
11
12         $twig = new Environment($loader, [
13             // En production : mettre un dossier r el pour le
14             // cache
15             'cache' => __DIR__ . '/../var/cache/twig',
16             // En d veloppement : d sactiver le cache
17             // 'cache' => false,
18             'debug' => true,
19         ]);
20
21         // Ajouter l'extension de debug (permet {{ dump() }})
22         $twig->addExtension(new \Twig\Extension\DebugExtension())
23             ;
24
25         // Variable globale accessible dans tous les templates
26         $twig->addGlobal('app_name', 'Mon Application PHP');

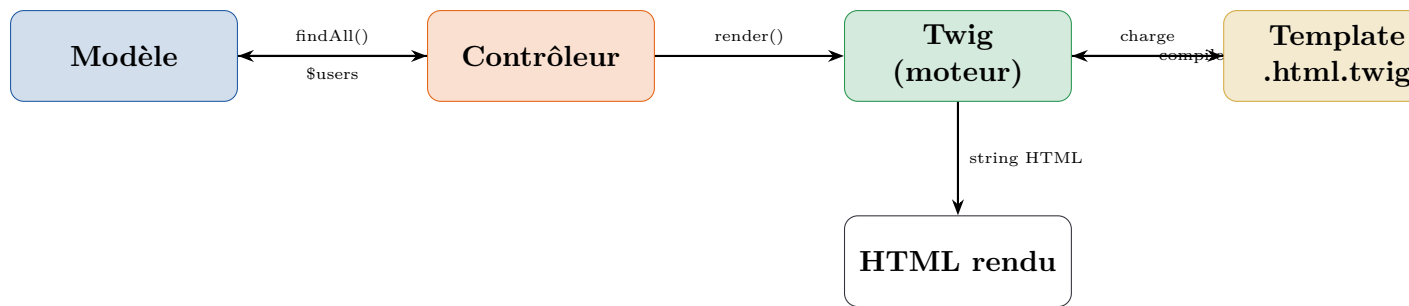
```

```
24
25     return $twig;
26 }
27 }
```

2.3.3. Contrôleur avec Twig —

</> Contrôleur utilisant Twig

```
1 <?php
2 namespace App\Controller;
3
4 use App\Model\UserRepository;
5 use Twig\Environment;
6
7 class UserController {
8     private UserRepository $repository;
9     private Environment $twig;
10
11     public function __construct(UserRepository $repository,
12                               Environment $twig) {
13         $this->repository = $repository;
14         $this->twig       = $twig;
15     }
16
17     public function list(): void {
18         $users = $this->repository->findAll();
19
20         // Rendu Twig : passe les données au template
21         echo $this->twig->render('users/list.html.twig', [
22             'users' => $users,
23             'title' => 'Liste des utilisateurs',
24         ]);
25     }
26
27     public function show(int $id): void {
28         $user = $this->repository->findById($id);
29
30         if ($user === null) {
31             http_response_code(404);
32             echo $this->twig->render('errors/404.html.twig');
33             return;
34         }
35
36         echo $this->twig->render('users/show.html.twig', [
37             'user' => $user,
38         ]);
39     }
40 }
```

3 Gestionnaire de dépendances — Composer

🎯 Objectifs

- [2] Expliquer le rôle d'un gestionnaire de dépendances
- [2] Reconnaître les éléments qui constituent un projet géré par Composer
- [3] Utiliser les commandes de Composer pour installer, mettre à jour et supprimer des dépendances

3.1 Rôle d'un gestionnaire de dépendances

📖 Définition

Composer est le gestionnaire de dépendances standard de PHP. Il permet de **déclarer**, **télécharger** et **gérer** les bibliothèques tierces utilisées par un projet, en résolvant automatiquement les conflits de version entre elles.

—
—
—
—

3.2 Structure d'un projet Composer

3.2.1. Le fichier `composer.json` —

📖 Définition

`composer.json` est le **fichier de configuration** du projet Composer. Il déclare les dépendances, les métadonnées du projet, et la configuration de l'autoloading. C'est le seul fichier que le développeur édite directement.

Listing 9 – `composer.json` — exemple complet

```
1 {  
2     "name": "mon-equipe/mon-projet",  
3     "description": "Application PHP MVC avec Twig",  
4     "type": "project",  
5     "license": "MIT",  
6     "require": {  
7         "php": ">=8.1",  
8         "twig/twig": "^3.0",
```

```

 9      "vlucas/phpdotenv": "^5.5"
10    },
11    "require-dev": {
12      "phpunit/phpunit": "^10.0",
13      "squizlabs/php_codesniffer": "^3.7"
14    },
15    "autoload": {
16      "psr-4": {
17        "App\\": "src/"
18      }
19    },
20    "autoload-dev": {
21      "psr-4": {
22        "Tests\\": "tests/"
23      }
24    },
25    "scripts": {
26      "test": "phpunit --testdox",
27      "cs-fix": "phpcbf --standard=PSR12 src/"
28    }
29  }

```

Clé

require
 require-dev
 autoload
 autoload-dev
 scripts

3.2.2. Le fichier `composer.lock` —



Point important

`composer.lock` contient les **versions exactes** de toutes les dépendances (et leurs dépendances transitives) installées. Il garantit que tous les membres de l'équipe et les serveurs de déploiement utilisent **exactement les mêmes versions**. Ce fichier doit être **versionné dans Git**.

3.2.3. Le répertoire `vendor/` —

```

vendor/
  autoload.php          # Point d'entr e de l'autoloading
  composer/             # Fichiers internes de Composer
    autoload_psr4.php   # Mapping namespace -> dossier

```

```
ClassLoader.php # Classe de chargement automatique
twig/
  twig/          # Code source de Twig
phpunit/
  phpunit/       # Code source de PHPUnit
```

💡 Bonne pratique

Règle d'or pour Git :

- ✓ Versionner : `composer.json` et `composer.lock`
- ✗ Ignorer : `vendor/` (le reconstruire avec `composer install`)

3.3 Commandes essentielles de Composer

3.3.1. Initialiser un projet —

```
# Cr er un composer.json interactivement
composer init

# Ou cr er directement depuis un mod le
composer create-project symfony/skeleton mon-projet
```

3.3.2. Installer les dépendances —

```
# Installer toutes les deps d clar es dans composer.json
# (utilise composer.lock si existant)
composer install

# Installer sans les deps de d veloppement (production)
composer install --no-dev

# Installer une d pendance et l'ajouter      composer.json
composer require twig/twig

# Installer en version contrainte
composer require twig/twig:^3.0

# Installer une d pendance de d veloppement
composer require --dev phpunit/phpunit
```

3.3.3. Mettre à jour les dépendances —

```
# Mettre      jour toutes les d pendencies (dans les contraintes)
# Met aussi      jour composer.lock
composer update

# Mettre      jour uniquement un package sp cifique
composer update twig/twig

# Voir les packages qui ont une mise      jour disponible
composer outdated
```

**Point important****Différence clé :**

- `composer install` : lit `composer.lock` et installe les versions exactes. À utiliser sur tous les postes / serveurs.
- `composer update` : résout les contraintes de `composer.json` et met à jour `composer.lock`. À utiliser avec précaution, après révision.

3.3.4. Supprimer une dépendance —

```
# Supprimer une dépendance
composer remove twig/twig

# Supprimer une dépendance de développement
composer remove --dev phpunit/phpunit
```

3.3.5. Autoloading et informations —

```
# Régénérer l'autoloader (après ajout de classes/namespaces)
composer dump-autoload

# Régénérer en mode optimisé pour la production
composer dump-autoload --optimize

# Afficher des informations sur une dépendance
composer show twig/twig

# Lister toutes les dépendances installées
composer show

# Valider le composer.json
composer validate
```

3.3.6. Syntaxe des contraintes de version —**3.4 L'autoloading PSR-4**



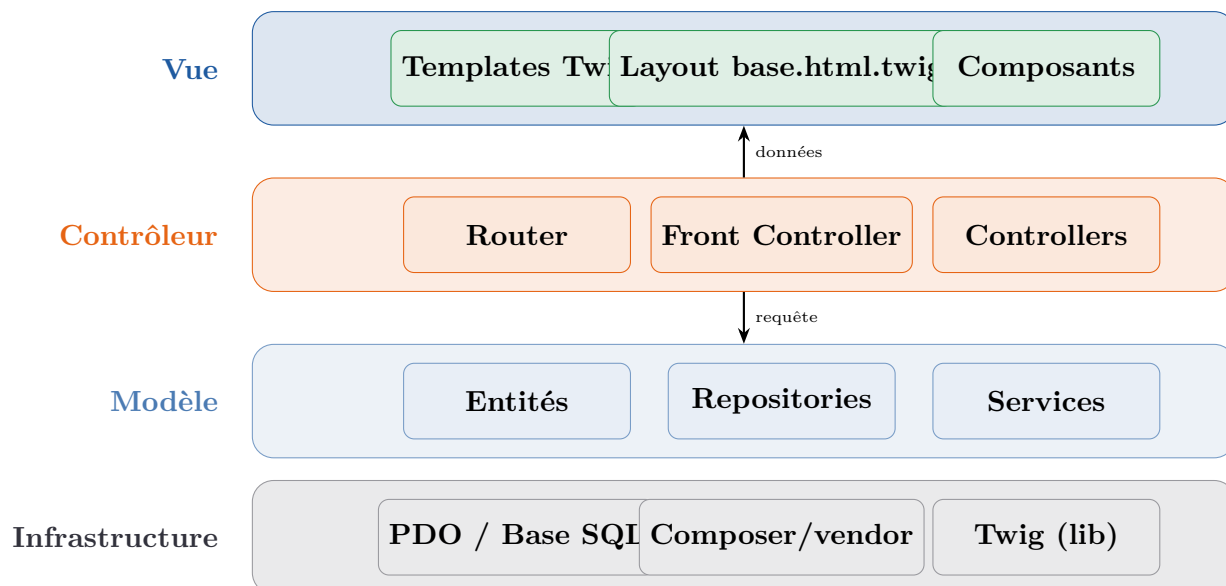
Définition

PSR-4 est une norme d'autoloading PHP qui définit une correspondance entre les **namespaces** et la **structure de fichiers**. Avec PSR-4, on n'a plus besoin d'écrire un seul **require** dans son code.

```
1 <?php
2 // public/index.php
3 require_once __DIR__ . '/../vendor/autoload.php';
4
5 // D sormais, toutes les classes App\* sont charg es automatiquement
6 use App\Controller\UserController;
7 use App\Model\UserRepository;
8 // ...
```

4 Synthèse : Projet PHP complet MVC + Twig + Composer

4.1 Architecture globale



4.2 Récapitulatif des commandes Composer

4.3 Récapitulatif des fonctionnalités Twig

 Bonne pratique**Pour aller plus loin :**

- **Symfony** : framework PHP basé sur MVC qui intègre nativement Twig, Composer, et des composants robustes (Routing, DI Container, ORM Doctrine...)
- **PHPUnit** : framework de tests unitaires à coupler avec votre architecture MVC pour tester les Modèles et Contrôleurs
- **Packagist** (packagist.org) : dépôt officiel des packages Composer — plus de 350 000 packages disponibles